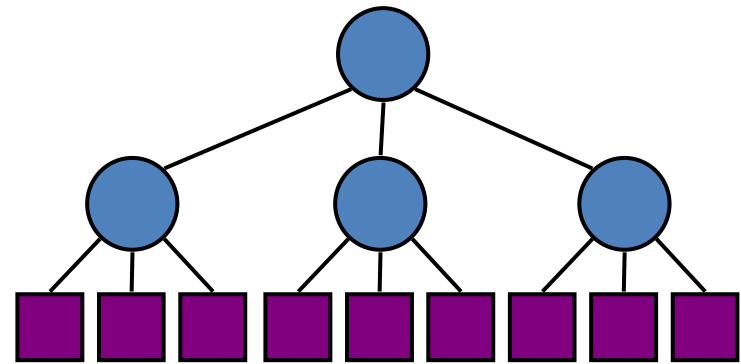


Divide-and-Conquer Technique: Merge Sort, Quick Sort

Divide-and-Conquer

- **Divide-and-Conquer** is a general algorithm design paradigm:
 - **Divide** the problem into a number of subproblems that are smaller instances of the same problem
 - **Conquer** the subproblems by solving them recursively
 - **Combine** the solutions to the subproblems into the solution for the original problem
- The base case for the recursion are subproblems of constant size
- Analysis can be done using **recurrence equations**



Merge Sort and Quick Sort

Two well-known sorting algorithms adopt this divide-and-conquer strategy

- Merge sort
 - Divide step is trivial – just split the list into two equal parts
 - Work is carried out in the conquer step by merging two sorted lists
- Quick sort
 - Work is carried out in the divide step using a pivot element
 - Conquer step is trivial

Merge Sort: Algorithm

MERGE-SORT(A, p, r)

1 **if** $p < r$

2 **then** $q \leftarrow \lfloor (p + r)/2 \rfloor$

3 MERGE-SORT(A, p, q)

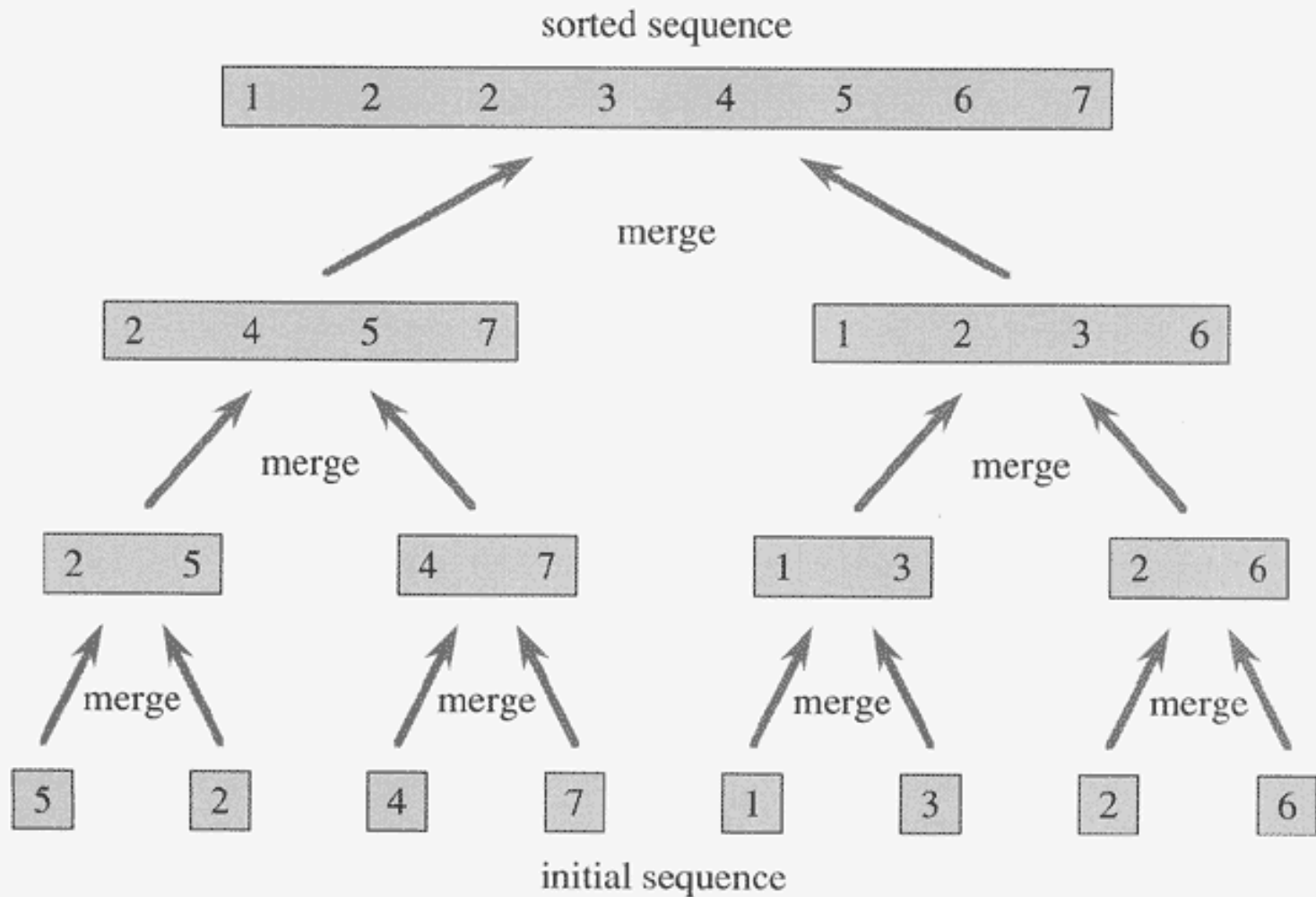
4 MERGE-SORT($A, q + 1, r$)

5 MERGE(A, p, q, r)

MERGE(A, p, q, r)

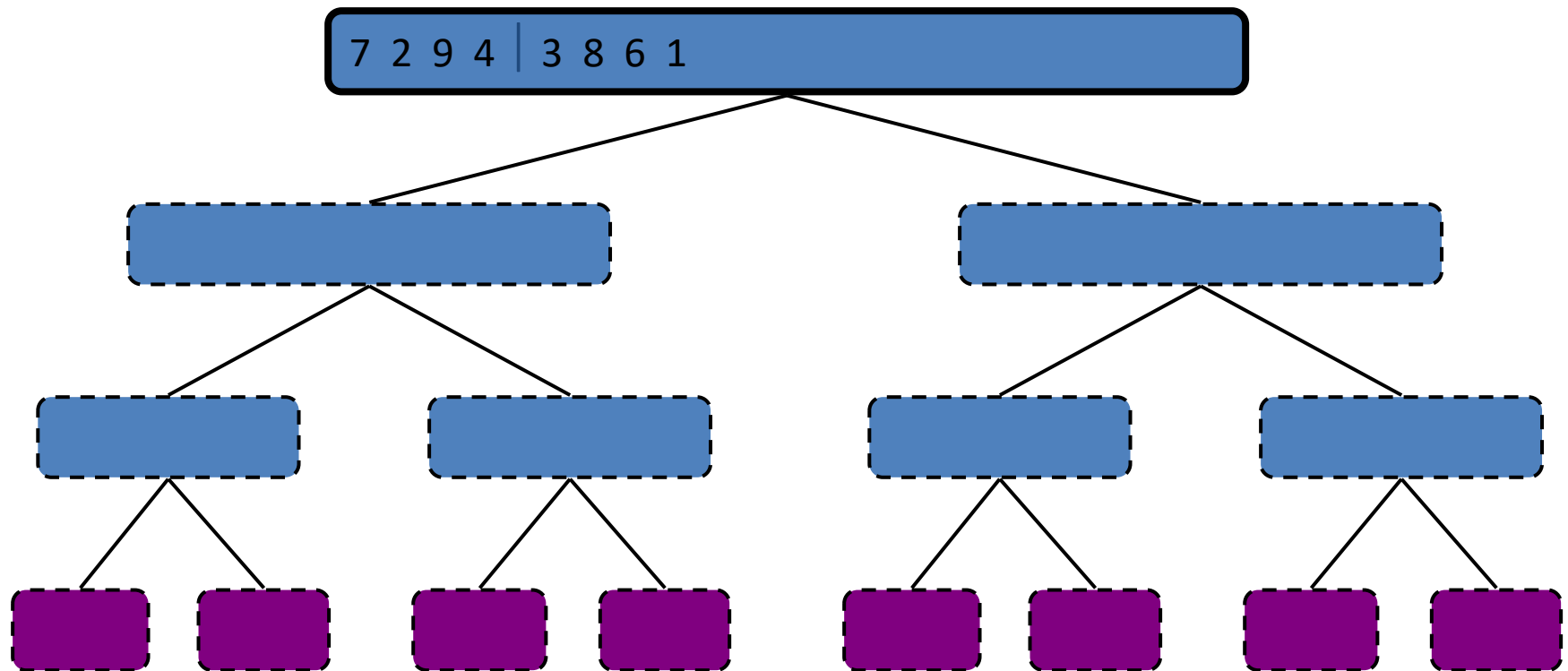
```
1   $n_1 \leftarrow q - p + 1$ 
2   $n_2 \leftarrow r - q$ 
3  create arrays  $L[1..n_1 + 1]$  and  $R[1..n_2 + 1]$ 
4  for  $i \leftarrow 1$  to  $n_1$ 
5      do  $L[i] \leftarrow A[p + i - 1]$ 
6  for  $j \leftarrow 1$  to  $n_2$ 
7      do  $R[j] \leftarrow A[q + j]$ 
8   $L[n_1 + 1] \leftarrow \infty$ 
9   $R[n_2 + 1] \leftarrow \infty$ 
10  $i \leftarrow 1$ 
11  $j \leftarrow 1$ 
12 for  $k \leftarrow p$  to  $r$ 
13     do if  $L[i] \leq R[j]$ 
14         then  $A[k] \leftarrow L[i]$ 
15              $i \leftarrow i + 1$ 
16         else  $A[k] \leftarrow R[j]$ 
17              $j \leftarrow j + 1$ 
```

Merge Sort: Example



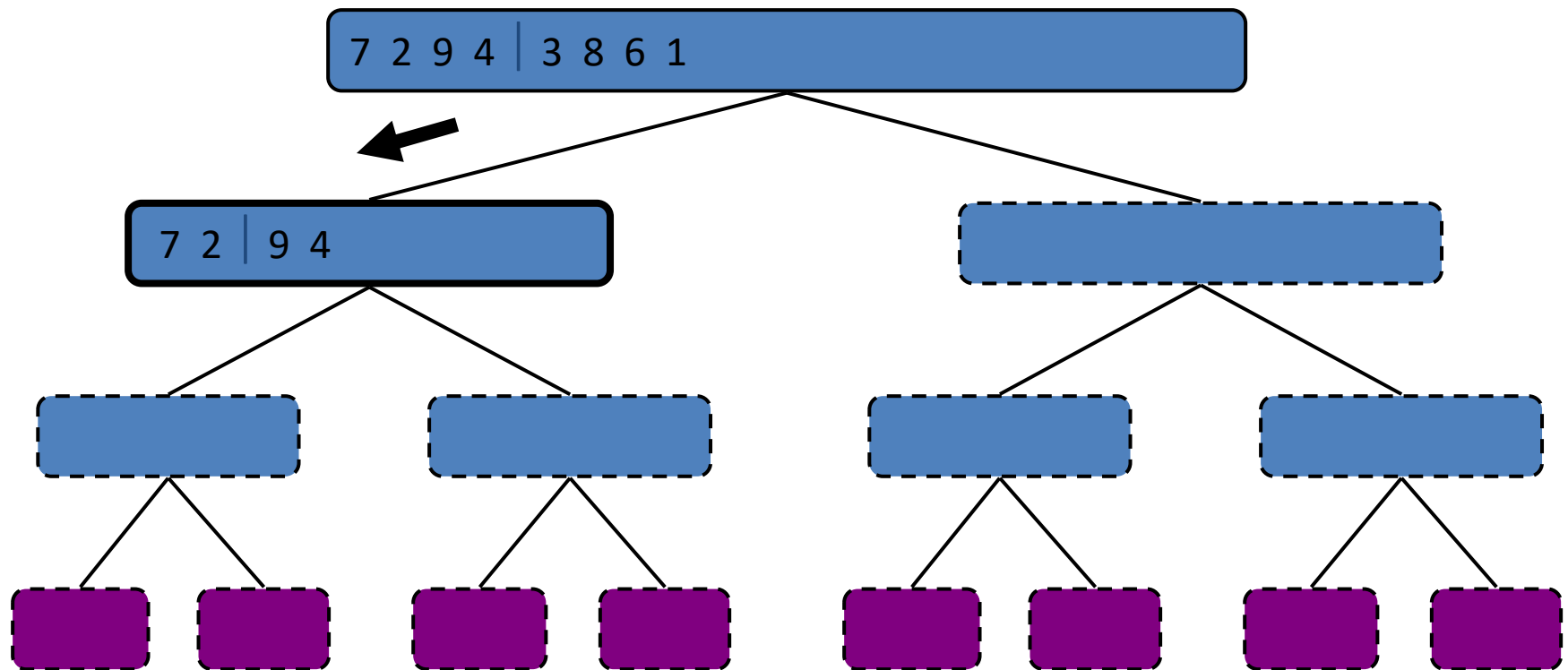
Execution Example

- Partition



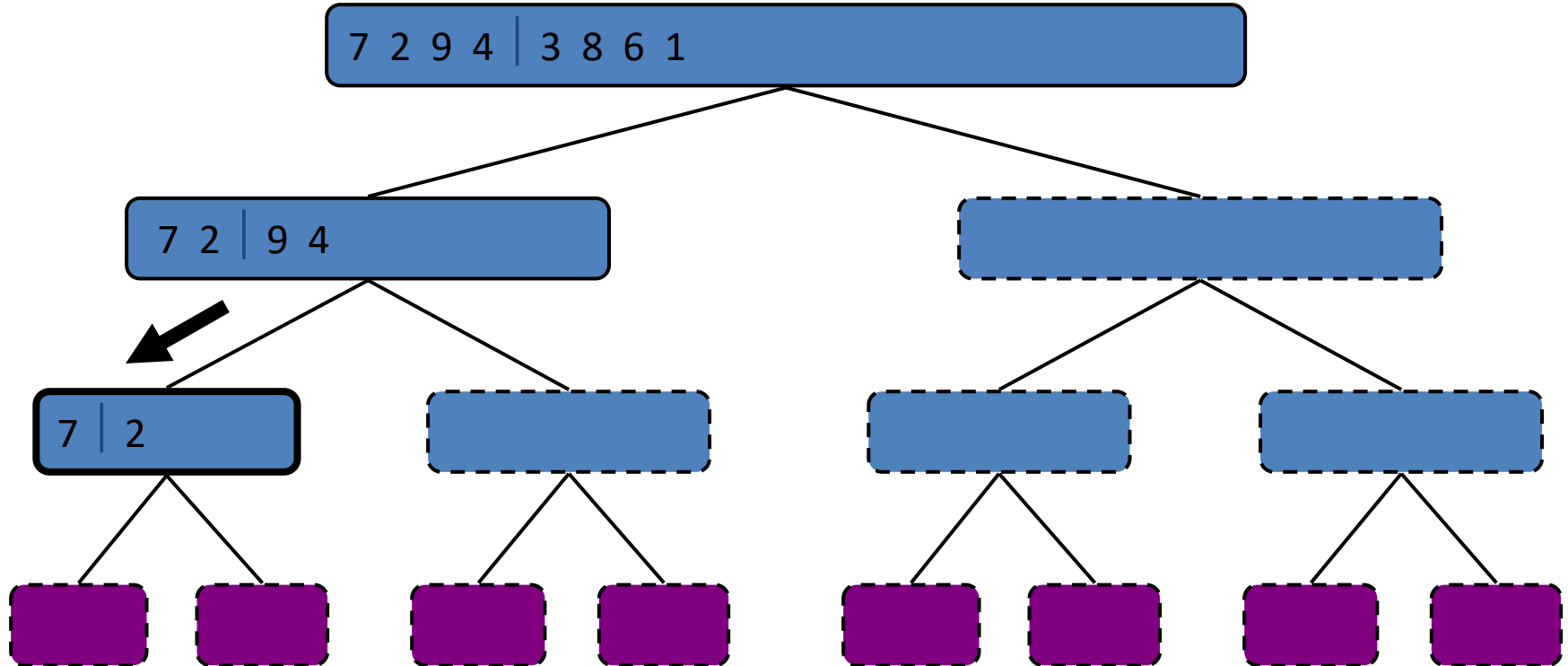
Execution Example (cont.)

- Recursive call, partition



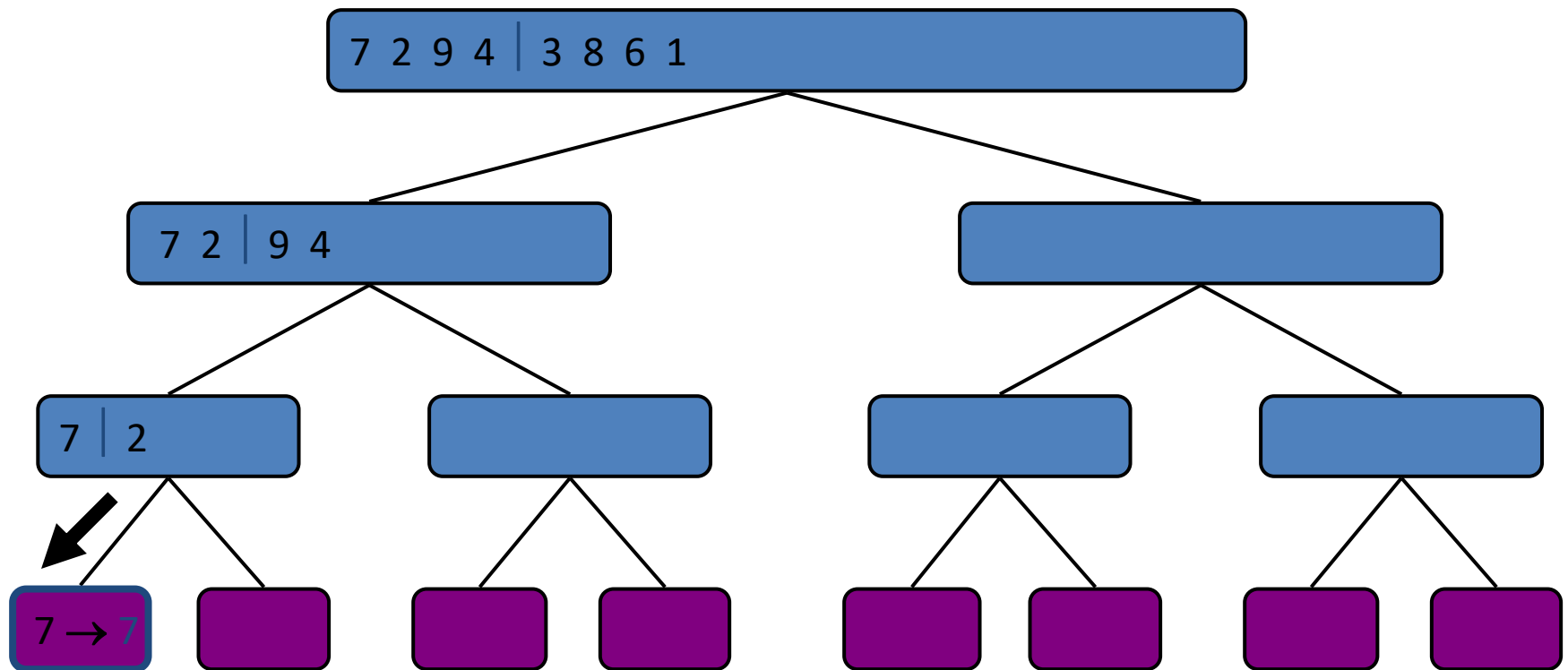
Execution Example (cont.)

- Recursive call, partition



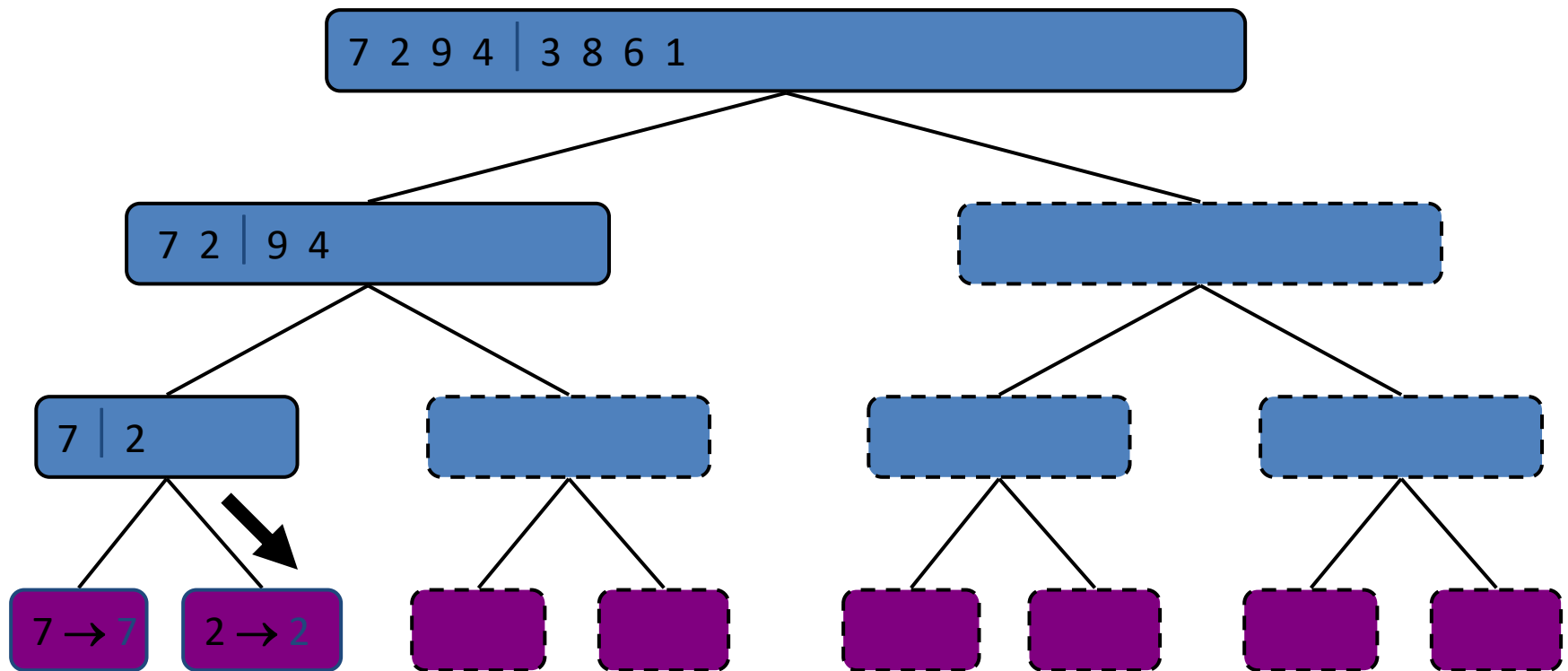
Execution Example (cont.)

- Recursive call, base case



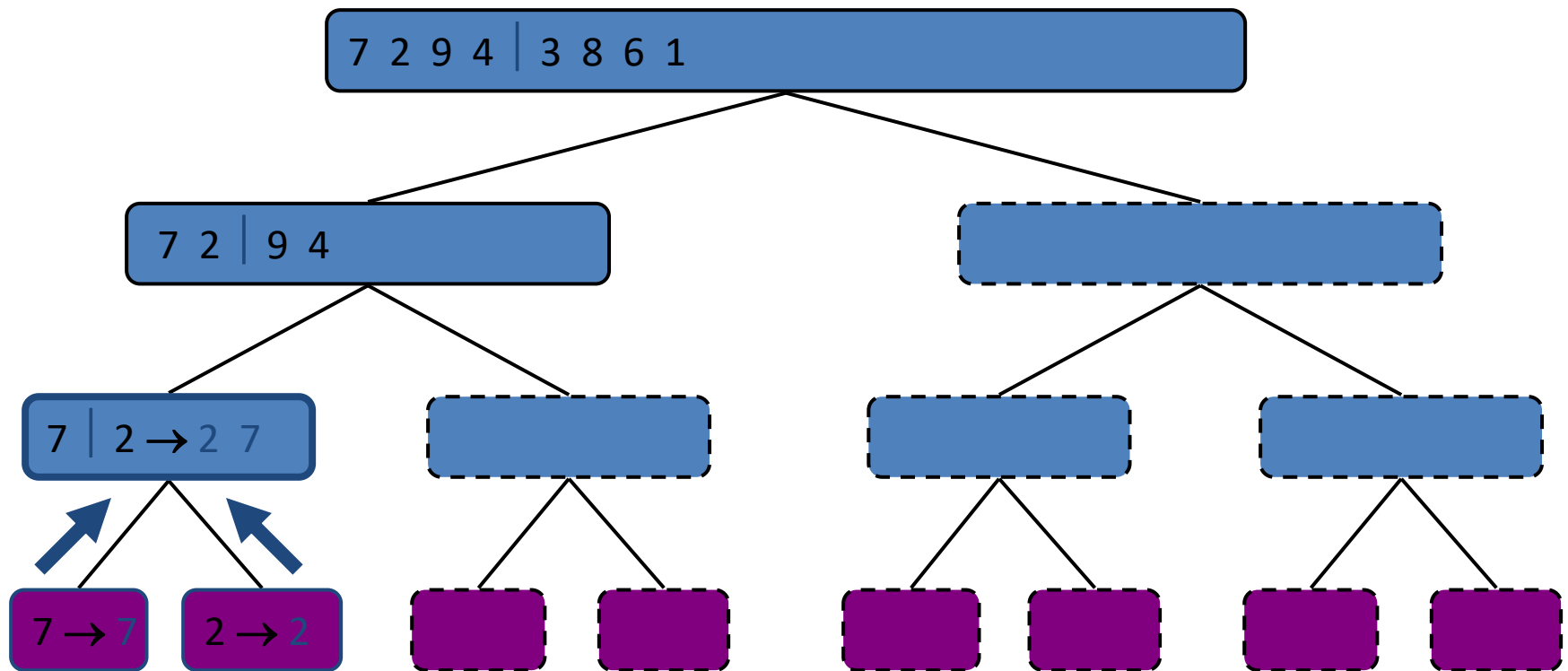
Execution Example (cont.)

- Recursive call, base case



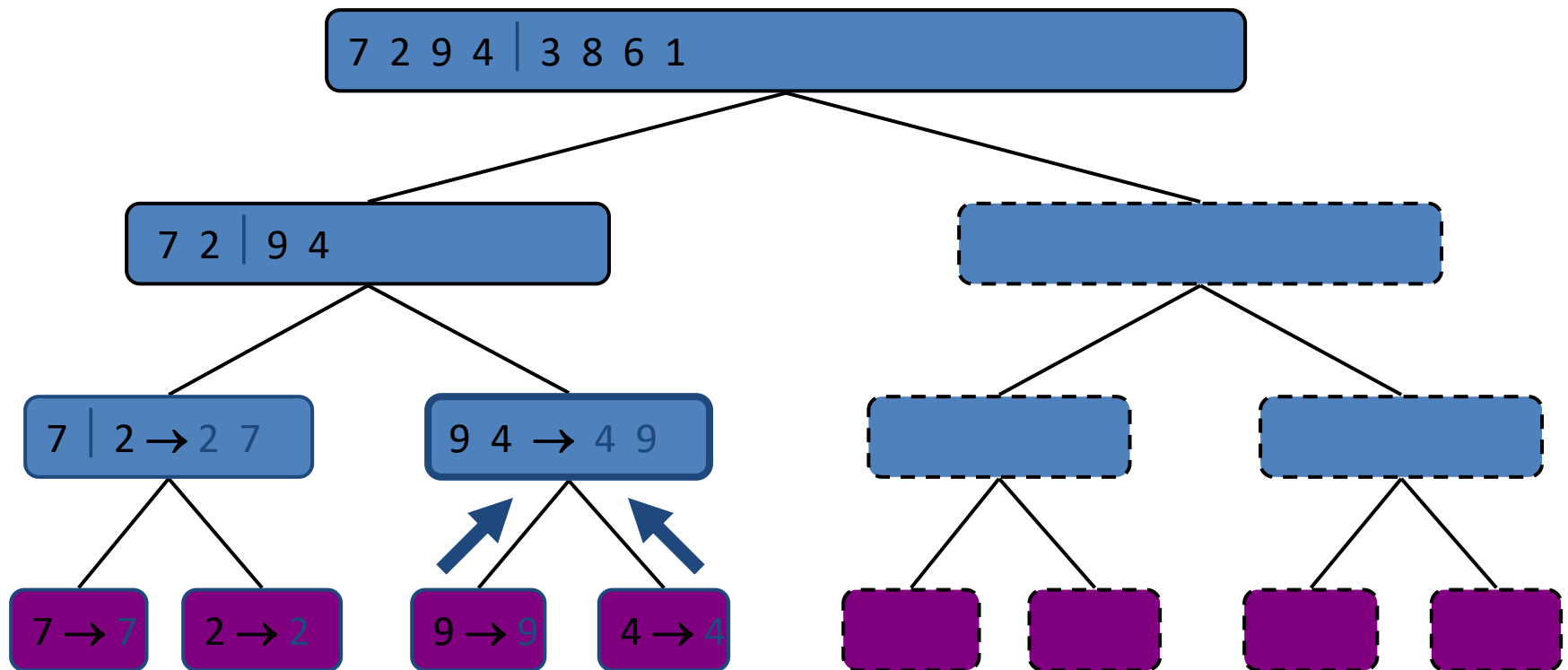
Execution Example (cont.)

- Merge



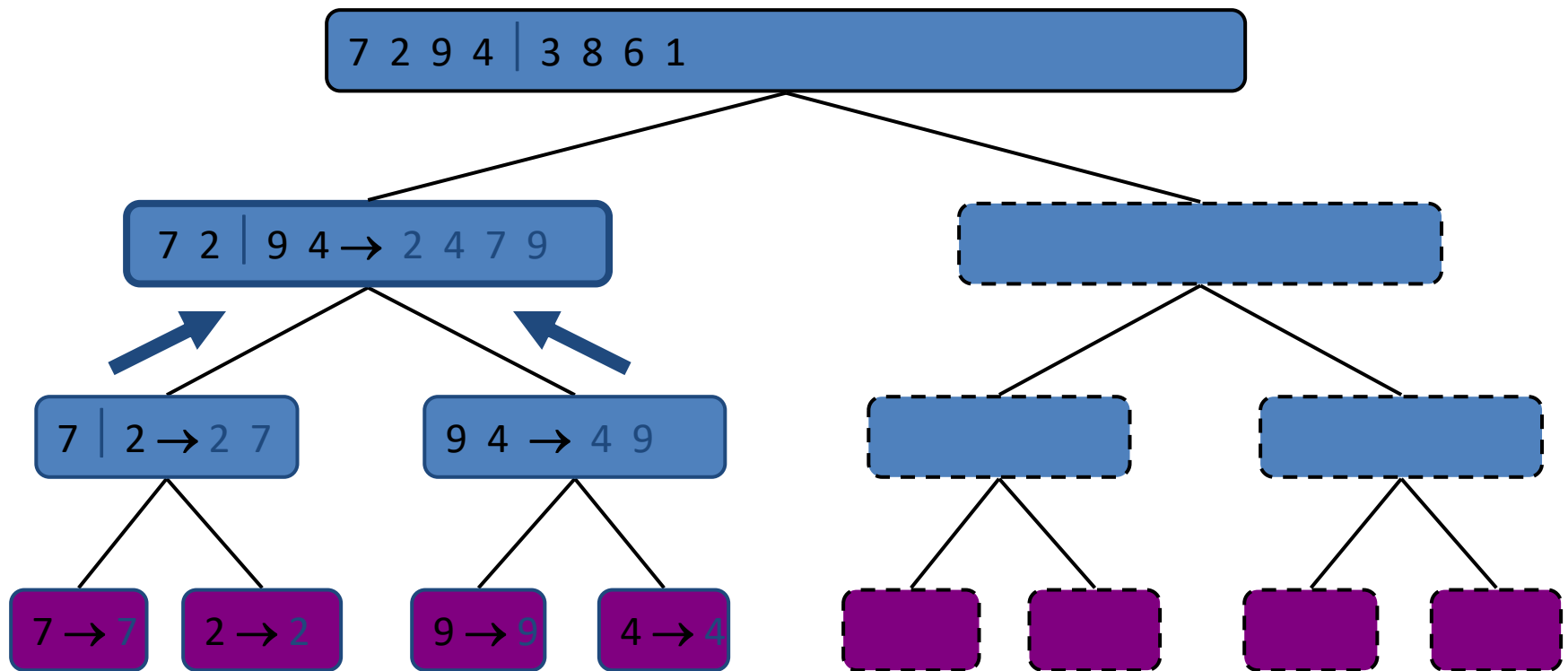
Execution Example (cont.)

- Recursive call, ..., base case, merge



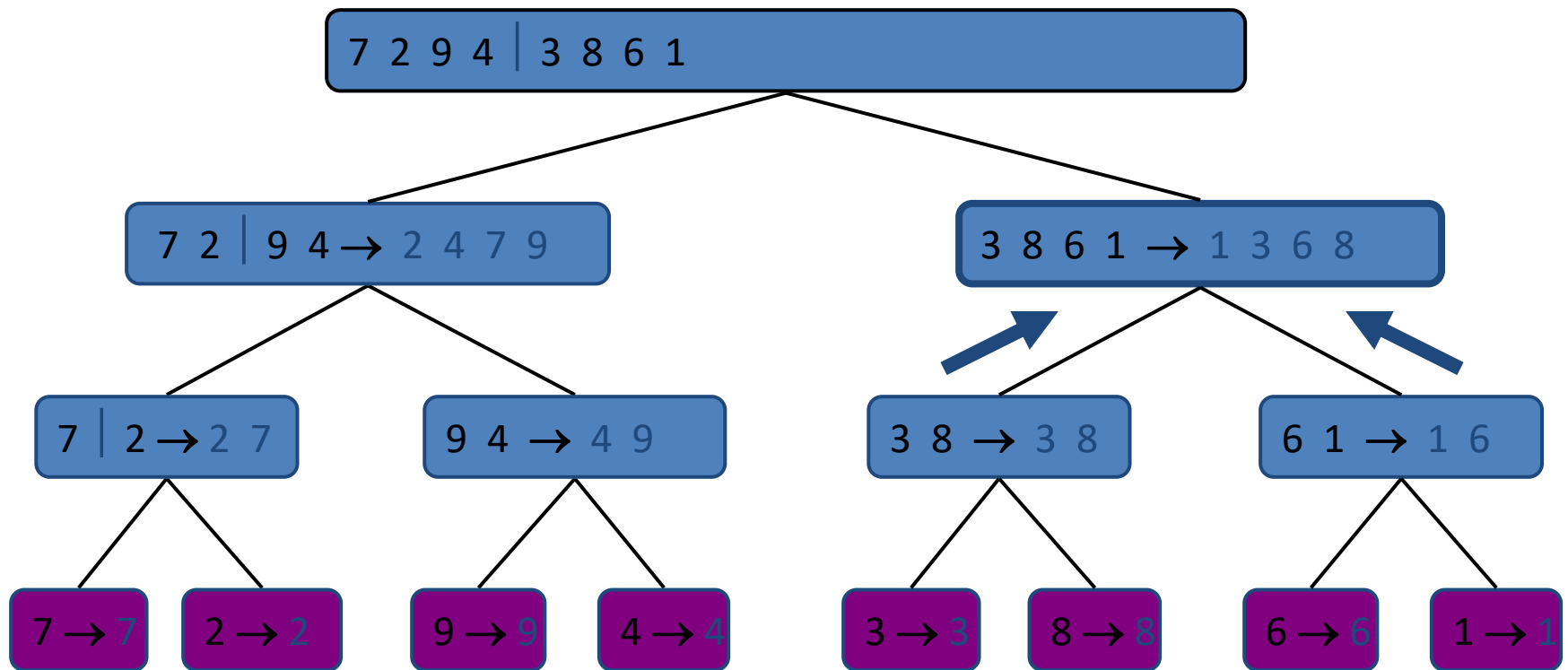
Execution Example (cont.)

- Merge



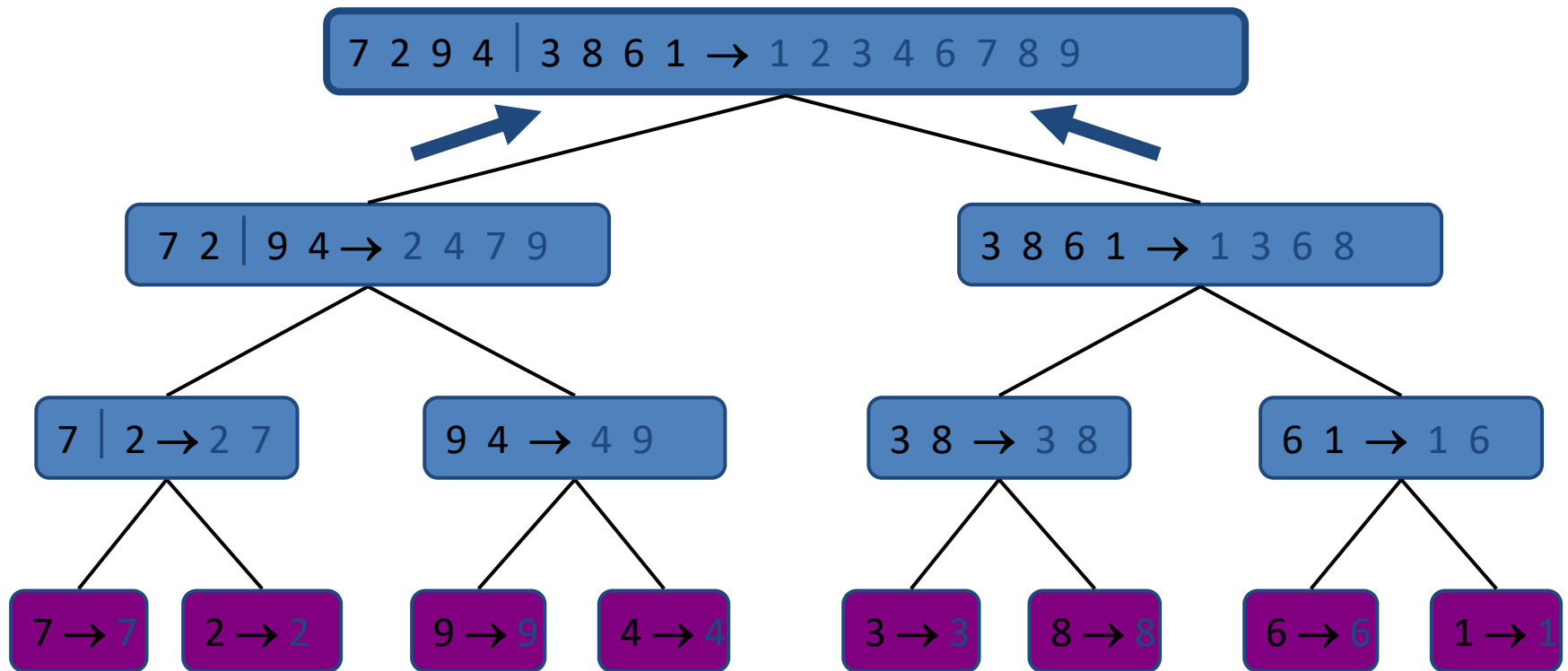
Execution Example (cont.)

- Recursive call, ..., merge, merge



Execution Example (cont.)

- Merge



Merge Sort: Running Time

The recurrence for the worst-case running time $T(n)$ is

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases}$$

equivalently

$$T(n) = \begin{cases} c & \text{if } n = 1 \\ 2T(n/2) + cn & \text{if } n > 1 \end{cases}$$

Solve this recurrence by

- (1) iteratively expansion
- (2) using the recursion tree

Merge Sort: Running Time (Iterative Expansion)

$$\begin{aligned}T(n) &= 2T(n/2) + bn \\&= 2(2T(n/2^2)) + b(n/2) + bn \\&= 2^2T(n/2^2) + 2bn \\&= 2^3T(n/2^3) + 3bn \\&= 2^4T(n/2^4) + 4bn \\&= \dots \\&= 2^iT(n/2^i) + ibn\end{aligned}$$

◆ Note that base, $T(1) = b$, case occurs when $2^i = n$.

That is, $i = \log n$.

◆ So,

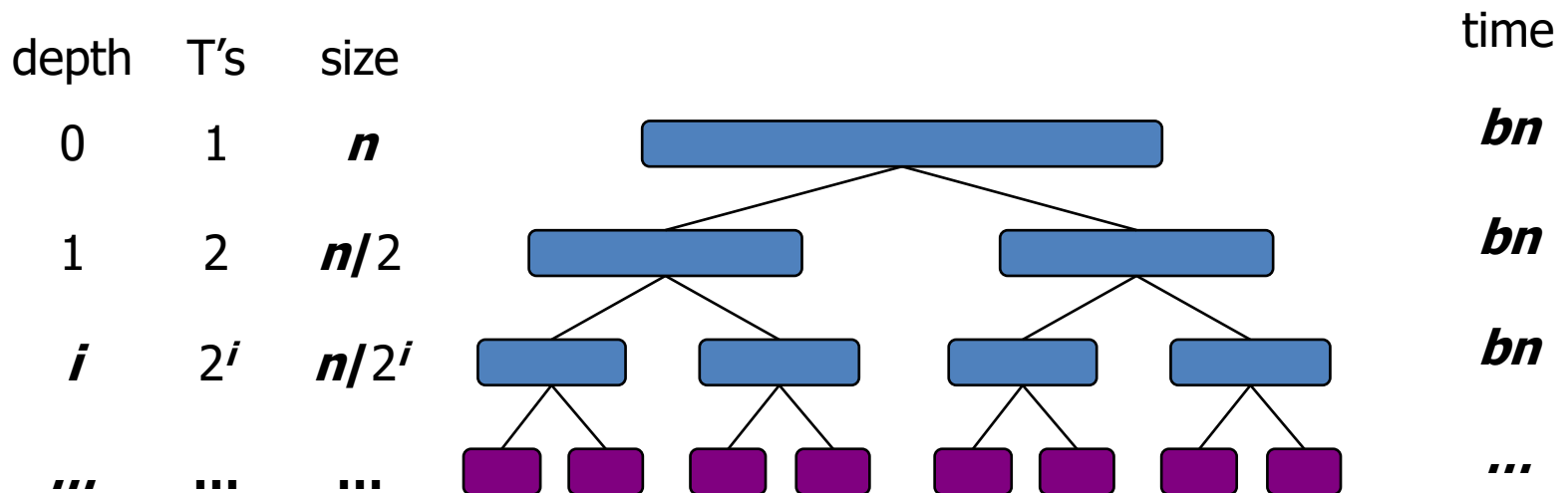
$$T(n) = bn + bn \log n$$

◆ Thus, $T(n)$ is $O(n \log n)$.

Merge Sort: Running Time (Recursion Tree)

- Draw the recursion tree for the recurrence relation and look for a pattern:

$$T(n) = \begin{cases} b & \text{if } n < 2 \\ 2T(n/2) + bn & \text{if } n \geq 2 \end{cases}$$



Total time = ***bn*** + ***bn*** log ***n***
(last level plus all previous levels)

Quick Sort: Algorithm

- Another divide-and-conquer algorithm
 - The array $A[p..r]$ is *partitioned* into two non-empty subarrays $A[p..q]$ and $A[q+1..r]$
 - Invariant: All elements in $A[p..q]$ are less than all elements in $A[q+1..r]$
 - The subarrays are recursively sorted by calls to quicksort
 - Unlike merge sort, no combining step: two subarrays form an already-sorted array

Quick Sort: Algorithm

QUICKSORT(A, p, r)

```
1  if  $p < r$ 
2      then  $q \leftarrow \text{PARTITION}(A, p, r)$ 
3          QUICKSORT( $A, p, q - 1$ )
4          QUICKSORT( $A, q + 1, r$ )
```

PARTITION(A, p, r)

```
1   $x \leftarrow A[r]$ 
2   $i \leftarrow p - 1$ 
3  for  $j \leftarrow p$  to  $r - 1$ 
4      do if  $A[j] \leq x$ 
5          then  $i \leftarrow i + 1$ 
6              exchange  $A[i] \leftrightarrow A[j]$ 
7  exchange  $A[i + 1] \leftrightarrow A[r]$ 
8  return  $i + 1$ 
```

Quick Sort: Algorithm (Partition)

- Clearly, all the actions take place in the **partition()** function
 - Rearranges the subarrays in place
 - End result:
 - Two subarrays
 - All values in first subarray $\leq$ all values in the second
 - Returns the index of the “pivot” element separating the two subarrays

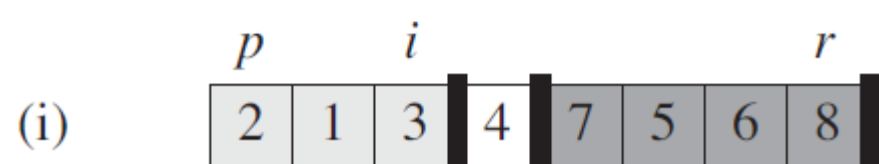
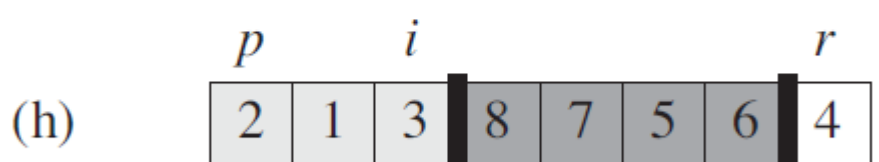
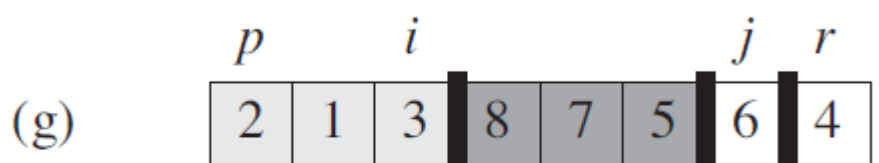
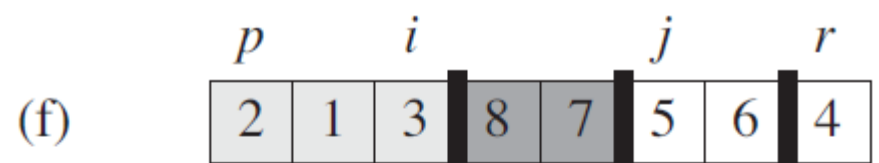
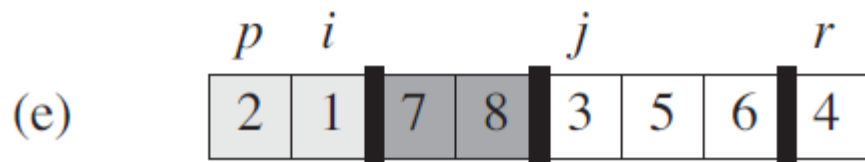
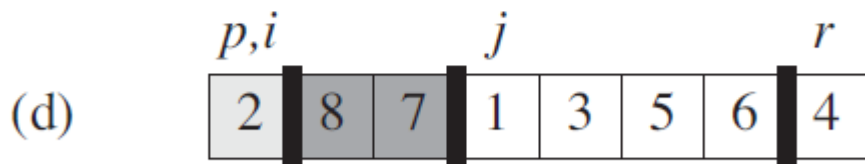
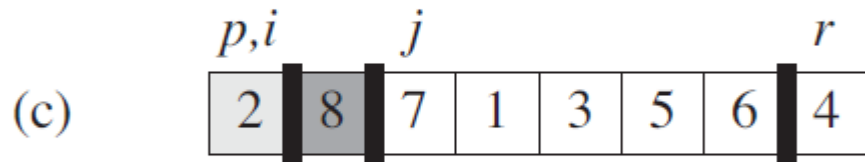
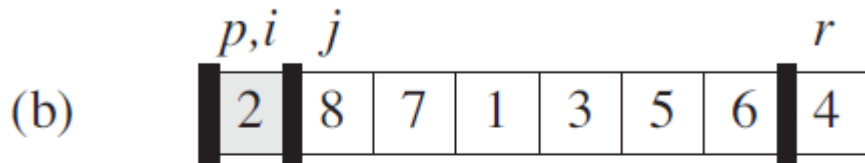
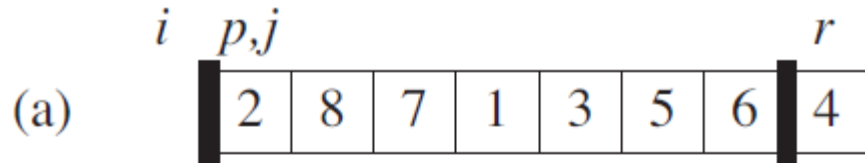
Quick Sort: Algorithm (Partition)

- The best practical algorithm.
- It applies the divide and conquer paradigm.
- Quicksort(A, p, r)
 1. if $p < r$
 2. $q = \text{Partition}(A, p, r)$ [find a pivot element at q and partition the array into three parts: left part smaller than pivot, pivot itself and right part larger than pivot.]
 3. Quicksort($A, p, q - 1$)
 4. Quicksort($A, q + 1, r$)

Quick Sort: Algorithm (Partition)

- Partition(A, p, r)
 $x = A[r]$
 $i = p - 1$
 for $j = p$ **to** $r - 1$
 if $A[j] \leq x$
 $i = i + 1$
 exchange $A[i]$ with $A[j]$
 exchange $A[i + 1]$ with $A[r]$
 return $i + 1$

Quick Sort: Algorithm



Quick Sort: Algorithm

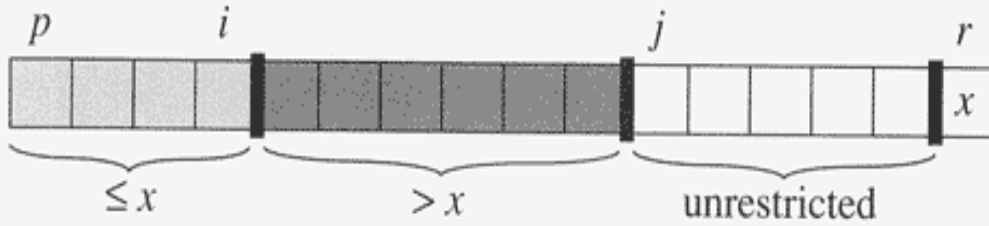
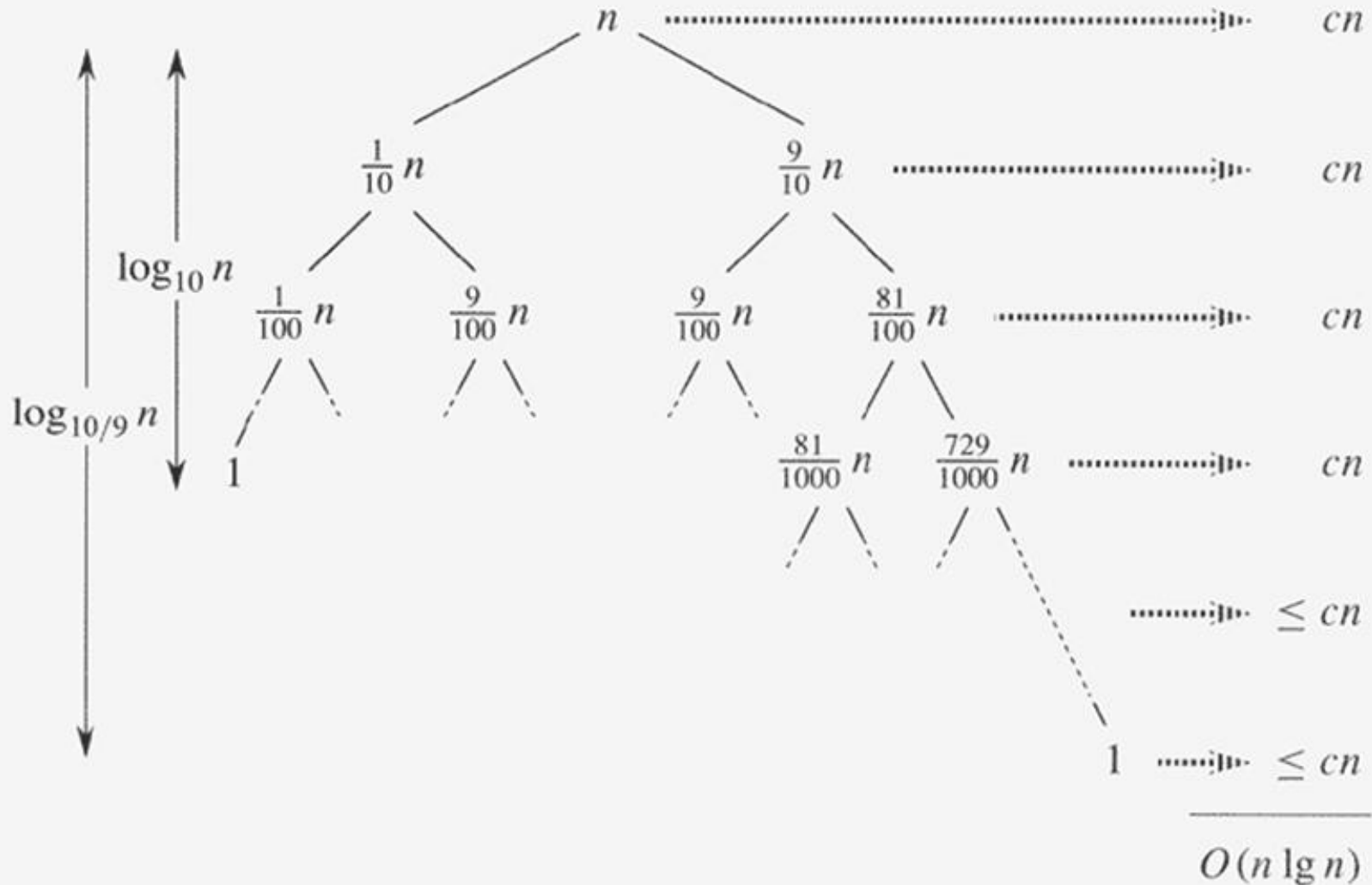


Figure 7.2 The four regions maintained by the procedure PARTITION on a subarray $A[p..r]$. The values in $A[p..i]$ are all less than or equal to x , the values in $A[i+1..j-1]$ are all greater than x , and $A[r] = x$. The values in $A[j..r-1]$ can take on any values.

Quick Sort: Analysis



Performance

- **Best-case partitioning:**

Partitioning routine produces two subproblems, each of size no more than $n/2$, since one is of size $\lfloor n/2 \rfloor$ and one of size $\lfloor n/2 \rfloor - 1$.

The recurrence for the running time is:

$$T(n) = 2T(n/2) + \Theta(n)$$

This recurrence has the solution $T(n) = \Theta(n \lg n)$.

Performance

- **Worst-case partitioning:**

Partitioning routine produces one subproblem with $n - 1$ elements and one with 0 element.

The partitioning costs $\Theta(n)$ time. Since the recursive call on an array of size 0 just returns, $T(0) = \Theta(1)$

$$\begin{aligned} T(n) &= T(n-1) + T(0) + \Theta(n) \\ &= T(n-1) + \Theta(n) \end{aligned}$$

This recurrence has the solution $T(n) = \Theta(n^2)$.